

Solución numérica e implementación: de las ecuaciones del PRB 2010 a las seis figuras

Emanuel Orozco Gallego

31 de agosto de 2026

Resumen

Este documento describe, con el código real del paquete `fibonacci_tmm`, cómo se resuelve numéricamente el modelo de Reyes-Gómez *et al.*, Phys. Rev. B **81**, 153101 (2010), y cómo se generan las Figs. 1–6. La física de fondo está en la *Guía conceptual* hermana; aquí el hilo es el algoritmo: unidades, matriz de transferencia, barrido en frecuencia, detección de subbandas y trazado.

Índice

1. Flujo global	3
2. Arquitectura del código	3
3. Unidades	3
4. De YAML a SuperlatticeSpec	4
5. Fibonacci en código	4
6. Materiales	4
7. Cinemática: n , q , Q , χ	4
8. Matriz de una capa	5
9. Semitraza: producto y recurrencia	5
10. Barrido de dispersión	5
11. Detección de modos plasmon-polaritón	6
12. Cómo se dibuja $k(\nu)$ sin manchas	6
13. Pipeline de cada figura	6
13.1. Figura 1	6
13.2. Figura 2	6
13.3. Figura 3	6
13.4. Figura 4	6
13.5. Figura 5	7
13.6. Figura 6	7
14. Regularizaciones numéricas	8

15.Tests automáticos	8
16.Cómo reproducir las gráficas	8
17.Decisiones que no están en el paper	9
18.Mapa de scripts a funciones	9

1. Flujo global

El cálculo no integra Maxwell en el tiempo. Para cada terna (m, θ, ν) evalúa la semitraza $R_m(\omega, \theta)$ y decide si esa frecuencia es banda o gap.

1. Un YAML en `configs/figure_0N.yaml` fija a , b , $\omega_e/2\pi$, $\omega_m/2\pi$, polarización y el barrido.
2. `params.spec_from_mapping` convierte milímetros y GHz a SI.
3. `scan_dispersion` recorre ν , calcula R_m y kL_m/π .
4. Los scripts `scripts/tmm/figure_0N.py` dibujan y guardan PDF/PNG/SVG más los datos `.npz` y un `summary.json`.

Dos caminos independientes para R_m , que deben coincidir:

- Producto de matrices de capa: $T_m = M_N \cdots M_1$, $R_m = \frac{1}{2} \text{Tr}(T_m)$.
- Recurrencia de Kohmoto: R_0, R_1, R_2 analíticos y $R_m = 2R_{m-1}R_{m-2} - R_{m-3}$.

Por defecto las figuras usan `method="recurrence"` (más barato en m alto). El producto sirve de test.

2. Arquitectura del código

El paquete vive en `src/fibonacci_tmm/`.

Módulo	Rol numérico	Qué calcula
<code>constants.py</code>	c , GHz, mm, tolerancias	SI y umbrales
<code>fibonacci.py</code>	S_m, F_m, L_m	palabras y conteos
<code>materials.py</code>	Drude y aire	$\varepsilon(\omega)$, $\mu(\omega)$
<code>params.py</code>	YAML $\rightarrow$ SI	<code>SuperlatticeSpec</code>
<code>electromagnetics.py</code>	n, q, Q, χ	cinemática de la onda
<code>transfer_matrix.py</code>	M, T_m, R_m	TMM
<code>dispersion.py</code>	$\cos(kL) = R$, intervalos	barrido y bandas
<code>plasmon_modes.py</code>	recuento	subbandas en una ventana
<code>bandwidth.py</code>	$\nu_{\min}(\theta), \nu_{\max}(\theta)$	Fig. 6
<code>plotting.py</code>	estilo PRB	cortes de curva y relleno
<code>provenance.py</code>	metadatos	reproducibilidad

Los parámetros científicos no están quemados en los scripts: están en YAML.

3. Unidades

El núcleo trabaja en SI. Las figuras hablan GHz y mm.

```
# constants.py
SPEED_OF_LIGHT = 299_792_458.0 # m/s [IMPL; el paper no da c]
GHZ = 1.0e9
MM = 1.0e-3
BAND_ABS_R_TOLERANCE = 1.0e-12
SMALL_Q_ABS = 1.0e-14
```

Conversiones en `params.py`:

$$\omega = 2\pi\nu_{\text{GHz}} \times 10^9, \quad a_{\text{SI}} = a_{\text{mm}} \times 10^{-3}. \quad (1)$$

Ejemplo: $\omega_e/2\pi = 3 \text{ GHz} \Rightarrow \omega_e = 1.884955592 \times 10^{10} \text{ rad/s}$. Usar $\omega_e = 3 \times 10^9$ sin el 2π desplazaría todas las figuras.

`dispersion.omega_from_nu_ghz` es la única puerta de ν a ω en el barrido.

4. De YAML a SuperlatticeSpec

Cada figura tiene su YAML. El de la Fig. 2, abreviado:

```
layer_a_thickness_mm: 12.0
layer_b_thickness_mm: 12.0
omega_e_over_2pi_ghz: 3.0
omega_m_over_2pi_ghz: 3.0
polarization: TE
fibonacci_orders: [3, 4, 5, 6]
thetas_rad: [1.0471975511965976] # pi/3
frequency_min_ghz: 2.0
frequency_max_ghz: 4.0
frequency_points: 12000
plasmon_window_ghz: [2.0, 2.999]
```

`spec_from_mapping` construye aire ($\varepsilon = \mu = 1$) y el Drude con ω_e, ω_m ya en rad/s. La función `load_figure_config` de `scripts/_common.py` devuelve (`spec`, `dict`, `ruta`): el diccionario conserva ángulos, ventanas y número de puntos que el `spec` no necesita.

5. Fibonacci en código

`paper_fibonacci(m)` implementa $F_0 = F_1 = 1$, no $F_0 = 0$. `sequence(m)` concatena $S_m = S_{m-1}S_{m-2}$ con memoización (`lru_cache`). Una segunda rutina, `sequence_by_inflation`, aplica $B \rightarrow A, A \rightarrow AB$; los tests exigen que ambas den la misma palabra.

Conteos:

```
n_layers_a(m) = paper_fibonacci(m - 1) # F(m-1)
n_layers_b(m) = paper_fibonacci(m - 2) # F(m-2) = N_modos
cell_length = n_a * a + n_b * b
```

Para $m = 0, 1$ se usa $F_{-1} = 0, F_{-2} = 1$. `substitution_identity` comprueba $S_m(A, B) = S_{m-k}[S_{k+1}, S_k]$.

El producto de matrices recorre `sequence(m)` de izquierda a derecha y premultiplica: si $S_2 = AB$, el bucle hace $T \leftarrow M_A I$ y luego $T \leftarrow M_B T$, es decir $T = M_B M_A = T_0 T_1$, como el paper.

6. Materiales

Aire: ε y μ constantes. Drude: se rechaza $\omega = 0$ (polo). No hay γ .

```
eps = epsilon_0 - omega_e**2 / omega**2
mu = mu_0 - omega_m**2 / omega**2
```

$\nu_e = \omega_e / (2\pi\sqrt{\varepsilon_0})$ y análogo para ν_m . Con $\varepsilon_0 = \mu_0 = 1$ coinciden con $\omega_{e,m} / 2\pi$ en GHz.

7. Cinemática: n, q, Q, χ

Índice: $n = \sqrt{\mu}\sqrt{\varepsilon}$ (dos raíces principales de NumPy). Para Q se usa $n^2 = \mu\varepsilon$, que no necesita el signo de n .

```
q = (omega / c) * n_A * sin(theta) # omega/c restaurado
Q = sqrt( (omega/c)**2 * n**2 - q**2 ) # complejo
chi = mu si TE; chi = epsilon si TM
```

`sqrt` complejo cubre ondas propagantes (Q casi real) y evanescentes (Q casi imaginario). `cos(Qd)` y `sin(Qd)` de NumPy aceptan argumento complejo y reproducen `cosh / sinh` sin un `if`.

8. Matriz de una capa

En un medio homogéneo, $E = A \cos(Qz) + B \sin(Qz)$. Exigir $\Psi(z + d) = M\Psi(z)$ con $\Psi = (E, \chi^{-1}E')$ da

$$M = \begin{pmatrix} \cos(Qd) & (\chi/Q) \sin(Qd) \\ -(Q/\chi) \sin(Qd) & \cos(Qd) \end{pmatrix}. \quad (2)$$

Cuando $|Q| < \text{SMALL_Q_ABS}$, $\sin(Qd)/Q \rightarrow d$ se reemplaza por el desarrollo $d(1 - \delta^2/6)$ para no dividir por cero ($Q = 0$ es el umbral de corte).

`layer_matrix` vectoriza sobre el barrido de ω : el array tiene forma $(2, 2, N_\omega)$. El producto es `einsum("ij...,jk...->ik...")`.

Comprobaciones analíticas (tests):

- $\det M = 1$.
- $\frac{1}{2}\text{Tr}(M_B) = R_0 = \cos(Q_B b)$.
- $\frac{1}{2}\text{Tr}(M_A) = R_1 = \cos(Q_A a)$.
- $\frac{1}{2}\text{Tr}(M_B M_A) = R_2$ (ecuación 9 del paper).

9. Semitraza: producto y recurrencia

`analytic_r0_r1_r2` escribe (7)–(9) con χ en lugar de μ , para que TM sea el mismo código. `semitrace_by_recurrence` itera

$$R \leftarrow 2R_{m-1}R_{m-2} - R_{m-3} \quad (3)$$

desde $m = 3$ hasta el orden pedido.

`semitrace_by_product` hace $\frac{1}{2}\text{Tr}(T_m)$ con el producto de capas. Los tests piden acuerdo relativo $\sim 10^{-8}$ hasta $m = 7$.

También existe `cell_matrix_by_recurrence_blocks`: $T_m = T_{m-2}T_{m-1}$ con $T_0 = M_B$, $T_1 = M_A$, para contrastar el orden de multiplicación.

10. Barrido de dispersión

Función central: `dispersion.scan_dispersion`.

1. $\omega_i = 2\pi\nu_i \times 10^9$ en una malla uniforme de ν .
2. Se marcan polos de Drude $|\omega - \omega_e| < 10^{-6}\omega_e$ o igual con ω_m ; esos puntos se excluyen de las bandas (Ψ no está definido si $\mu = 0$ o $\varepsilon = 0$).
3. Se evalúa $R_m(\omega_i)$ por recurrencia o producto.
4. Máscara de banda: $|R| \leq 1 + 10^{-12}$. El entero `clipped_near_unit` cuenta cuántos puntos cayeron en $(1, 1 + 10^{-12}]$ (redondeo, no recorte silencioso del gap).
5. Donde hay banda, $kL_m/\pi = \arccos(\text{Re } R)/\pi \in [0, 1]$. En gaps se deja NaN, no se inventa un k .

El objeto `DispersionScan` guarda ν , ω , R , $|R|$, la máscara, kL_m/π , el método y cuántos puntos se cliparon.

`allowed_intervals` recorre la máscara y agrupa componentes conexas: cada racha de `True` es un `FrequencyInterval` ($\nu_{\text{mín}}, \nu_{\text{máx}}, n_{\text{puntos}}$), con $\Delta\nu = \nu_{\text{máx}} - \nu_{\text{mín}}$.

11. Detección de modos plasmon-polaritón

`detect_plasmon_modes` filtra los intervalos que intersectan una ventana $[\nu_a, \nu_b]$ y tienen al menos `min_points` muestras (para no contar un punto aislado de malla como modo). El número esperado es $n_{layers_b}(m) = F_{m-2}$.

`merge_touching_intervals` une dos intervalos si el hueco entre ellos es menor que `gap_ghz` (artefacto de un punto prohibido en medio de una banda).

En la Fig. 2 la ventana es $[2.0, 2.999]$ GHz: se corta justo debajo del polo $\nu_m = 3$ GHz.

12. Cómo se dibuja $k(\nu)$ sin manchas

Un `plot(k, nu)` ingenuo une puntos consecutivos en ω . Cuando k salta de ~ 0 a ~ 1 al cruzar un gap, Matplotlib traza una recta horizontal que rellena el panel (el artefacto de la primera Fig. 1).

`plot_dispersion_branches` hace tres cosas:

1. Pone NaN donde no hay banda.
2. Cierra huecos de *un* solo punto si los vecinos en k están cerca (`_fill_isolated_nans`): quita un poro numérico en $k = 0$.
3. Inserta NaN si $|\Delta k| > 0.12$ (`_polyline_with_breaks`): corta saltos de zona de Brillouin.
4. Dibuja $+k$ y $-k$ para llenar $[-1, 1]$.

Estilos: S_3 rojo discontinuo, S_4 azul continuo, ticks hacia dentro.

13. Pipeline de cada figura

13.1. Figura 1

Script: `scripts/tmm/figure_01.py`. Config: $\nu_e = \nu_m = 3$ GHz, $m \in \{3, 4\}$, $\theta \in \{0, \pi/12, \pi/6, \pi/3\}$, 12000 puntos en $[0.15, 5]$ GHz (se evita $\nu = 0$, polo de Drude).

Para cada panel: `scan_dispersion` con `method="recurrence"`, `plot_dispersion_branches` con `S3_STYLE/S4_STYLE`, ejes $kL_m/\pi \in [-1, 1]$, $\nu \in [0, 5]$. Se guarda también la frecuencia donde se cierra el gap $\langle n \rangle = 0$: `zero_average_index_frequency_ghz`.

13.2. Figura 2

$\theta = \pi/3$, $\nu \in [2, 4]$ GHz, $m = 3, 4, 5, 6$. Después de dibujar se llama a `detect_plasmon_modes` en $[2, 2.999]$ GHz. El `summary.json` debe leer `detected = expected = 1, 2, 3, 5`. Línea horizontal a $\nu_m = 3$ GHz.

13.3. Figura 3

Igual que la Fig. 1 en geometría y ángulos, pero $\omega_m/2\pi = 1$ GHz. No se dibuja línea de referencia (el original no la tiene). La banda plana a 1 GHz aparece sola, como un segmento casi horizontal, cuando $\theta \neq 0$.

13.4. Figura 4

Cuatro ventanas densas (16000 puntos cada una):

Panel	m	θ	ν (GHz)
(a)	3	$\pi/12$	0.994–1.002
(b)	3	$\pi/3$	0.94–1.02
(c)	4	$\pi/12$	0.994–1.002
(d)	4	$\pi/3$	0.94–1.02

Se detectan intervalos y se informa $\Delta\nu$. Valores de esta implementación: 4.51 MHz ($m = 3$, $\pi/12$) y 33.1 MHz ($m = 3$, $\pi/3$). Convergencia (`scripts/analyze_convergence.py`): el ancho a $\pi/3$, $m = 3$, pasa de 0.03299 GHz (500 puntos) a 0.03310 GHz (16000); estable desde 2000 puntos ($< 0.2\%$).

13.5. Figura 5

Para $\theta = \pi/12$ y $\pi/3$, y cada $m = 2, \dots, 8$: escanear, extraer intervalos en la ventana, dibujar un palo vertical $[m, m] \times [\nu_{\min}, \nu_{\max}]$ con `BRANCH_COLORS[índice]` (negro, rojo, verde, azul, ... de menor a mayor ν). Eje x : “Fibonacci order”. Eje y : “bandwidth (GHz)”, rotulado como el impreso; los valores son frecuencias. Aquí no se usa `merge_touching_intervals`: a $\theta = \pi/12$ y $m = 7$ hay gaps físicos ($|R_m| > 1$) de apenas 0.4 kHz, y fusionarlos reduciría el conteo por debajo de F_{m-2} . Con la detección directa se obtienen 1, 1, 2, 3, 5, 8, 13 subbandas para $m = 2, \dots, 8$ en ambos ángulos, estable con 20 000, 80 000 y 320 000 puntos.

13.6. Figura 6

No es un $\Delta\nu(\theta)$ en líneas. Algoritmo:

1. Malla de $\theta \in [0, \pi/3]$ (41 puntos). La malla en ν une 60 000 puntos logarítmicos en $\nu_m - \nu \in [10^{-10}, 0.1]$ GHz y 250 000 puntos uniformes en $[0.95, \nu_m)$ (paso de 0.2 kHz). Hace falta porque cerca de $\theta = 0$ las subbandas quedan a $\sim 10^{-5}$ GHz de ν_m con anchos de apenas $\sim 5 \times 10^{-8}$ GHz, y hacia $\theta \approx \pi/12$ hay gaps físicos de 0.4 kHz. Una malla uniforme de 16 000 puntos cortada en $\nu_m - 10^{-4}$ perdía todas las subbandas por debajo de $\sim 3^\circ$ y parte de ellas hasta $\sim 20^\circ$.
2. `bandwidth_versus_angle` sin fusionar intervalos: en cada $\theta > 0$ se obtienen exactamente F_{m-2} subbandas conexas (el script avisa si algún ángulo no las tiene).
3. Cada borde se refina con Brent entre la última muestra con $|R_m| \leq 1$ y su vecina exterior, hasta donde $|R_m| = 1$ (`refine=True`). Sin este paso el ancho queda subestimado hasta en dos pasos de malla, lo que importa en las subbandas más estrechas: con $m = 7$ algunas perdían más del 1 % de su ancho. Lo mismo se hace con los bordes de la Fig. 5.
4. Como el número de subbandas no cambia con θ , la subbanda i -ésima contada desde abajo es la misma rama física en todos los ángulos y conserva su color.
5. A $\theta = 0$ el plasmón está en el gap: todas las ramas se clavan en $(\theta, \nu) = (0, \nu_m)$, como en el original.
6. `fill_between` por tramos contiguos (no se unen huecos con una diagonal).

Colores, de menor a mayor ν : negro, rojo, verde, azul, magenta, cian. Número de regiones para todo $\theta > 0$: 1, 1, 2, 3, 5, 8 = F_{m-2} para $m = 2, \dots, 7$.

14. Regularizaciones numéricas

Situación	Qué hace el código	Por qué
$\omega = 0$	<code>ZeroDivisionError</code>	Drude diverge
$\omega = \omega_e$ o ω_m	vecindad relativa 10^{-6} excluida	polo de ε o μ
$Q \rightarrow 0$	$\sin(Qd)/Q \rightarrow d(1 - \delta^2/6)$	corte geométrico
$ R = 1 + \varepsilon_{\text{maq}}$	banda si $\varepsilon \leq 10^{-12}$	redondeo, documentado
$\text{Im } R \neq 0$	se usa $\text{Re } R$ en arc cos	modelo sin pérdidas

A $\theta = 0$ con $\omega_e = \omega_m$, $|R_m| \leq 1$ en todo el barrido: no es un bug, es adaptación de impedancia con el aire.

15. Tests automáticos

`pytest tests` cubre:

- `test_fibonacci.py`: palabras, inflación, N_B , L_m , identidad de substitución.
- `test_materials.py`: ceros de Drude, ν_e y ν_m .
- `test_transfer_matrix.py`: $\det T = 1$, (7)–(9), recurrencia vs producto, y que TE coincide con TM a $\theta = 0$.
- `test_dispersion.py`: máscara de banda, frecuencia $\langle n \rangle = 0$.
- `test_validation.py`: $N_{\text{modos}} = F_{m-2}$ en las ventanas de las Figs. 2 y 4.
- `test_params.py`: YAML de cada figura.

Tres casos se omiten (`skip`) cuando $k > m$ en una identidad de Fibonacci que no aplica.

16. Cómo reproducir las gráficas

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pytest
python scripts/tmm/run_all.py
# o una sola: python scripts/tmm/figure_04.py
```

Salida de cada figura:

- `results/tmm/figure_0N/output/figure_0N.pdf` (y PNG, SVG).
- `results/tmm/figure_0N/data/*.npz`: ν , $|R|$, máscara, kL_m/π .
- `results/tmm/figure_0N/data/summary.json`: recuentos y anchos.

`make figures` (o `make tmm`) lanza además `scripts/tmm/convergence.py`. La verificación con ondas planas (`scripts/pwe/`, `scripts/comparison/`) está descrita en el documento *El método de expansión en ondas planas* y en el anexo *Comparación TMM vs PWE*.

Para TM: `polarization: TM` en el YAML (no hay figuras TM en el original; el código está listo). Para otro plasma: editar `omega_e_over_2pi_ghz` y `omega_m_over_2pi_ghz`. Para otro orden: `fibonacci_orders`.

17. Decisiones que no están en el paper

Todas etiquetadas en comentarios como implementación, no como dato original:

Decisión	Valor
q	$(\omega/c)n_A \sin \theta$
c	299792458 m/s
matriz de capa	$M(Q, d, \chi)$ unimodular
rama de Q	<code>sqrt</code> complejo principal
banda	$ R \leq 1 + 10^{-12}$
ancho	$\nu_{\text{máx}} - \nu_{\text{mín}}$ de cada componente conexas
Fig. 6	parámetros de Figs. 3–5, $m = 2 \dots 7$

No se calibró nada contra el dibujo impreso.

18. Mapa de scripts a funciones

Figura	Script	Núcleo
1	<code>tmm/figure_01.py</code>	<code>scan_dispersion + plot_dispersion_branches</code>
2	<code>tmm/figure_02.py</code>	<code>idem + detect_plasmon_modes</code>
3	<code>tmm/figure_03.py</code>	igual que 1, otro YAML
4	<code>tmm/figure_04.py</code>	ventanas densas + anchos
5	<code>tmm/figure_05.py</code>	<code>allowed_intervals</code> por m
6	<code>tmm/figure_06.py</code>	<code>bandwidth_versus_angle + fill_between</code>

El camino mínimo, para una sola curva, es:

```
from fibonacci_photonics.core.params import load_spec
from fibonacci_photonics.tmm.dispersion import scan_dispersion
from fibonacci_photonics.core.electromagnetics import Polarization
import numpy as np

spec = load_spec("configs/figure_02.yaml")
nu = np.linspace(2.0, 4.0, 4000)
scan = scan_dispersion(spec, m=3, theta=np.pi/3, nu_ghz=nu,
                      polarization=Polarization.TE)
# scan.allowed, scan.k_lm_over_pi, scan.nu_ghz
```

Eso es, en código, toda la física de $\cos(kL_m) = R_m$.

Referencias

- [1] E. Reyes-Gómez *et al.*, Phys. Rev. B **81**, 153101 (2010).
- [2] E. Reyes-Gómez *et al.*, EPL **88**, 24002 (2009).
- [3] M. Kohmoto, L. P. Kadanoff y C. Tang, Phys. Rev. Lett. **50**, 1870 (1983).